

// UCS MCP – BUILD BLUEPRINT V3.0 – UPDATED JUNE 2026

The **production-grade** Odoo AI bridge that costs almost nothing to run

Six AI clients supported. Four competitors analysed and outclassed. One Zero-Cost Intelligence Layer that answers 70% of queries from cache — no AI API call needed. LGPL-3, Odoo 19 native, OCA standards.

38+

MCP TOOLS

6

AI CLIENTS

~95%

API COST REDUCTION

5

AI PROVIDERS

LGPL-3

OPEN SOURCE

Odoo 19

NATIVE ONLY

CONTENTS

- 01 Why we built this
- 02 Supported AI clients
- 03 Competitor analysis
- 04 Feature matrix
- 05 Zero-Cost Intelligence Layer ★
- 06 System architecture
- 07 Development phases
- 08 Complete tool catalog
- 09 Technical decisions

CONTEXT

What problem we are solving

Claude Code, Claude Desktop, Cursor, Antigravity — all AI agents can call external tools via MCP. Odoo 19 has 800+ models covering every business process. The gap between them costs developers hours of JSON-RPC wiring and locks companies into SaaS AI tools that route ERP data through third-party servers and charge per query.

The correct answer: an MCP server baked into Odoo that (1) uses Odoo's ORM / record rules / ACLs natively, (2) pre-computes answers so most queries need zero external API calls, and (3) routes the remaining calls to the cheapest model that can answer correctly.

Why not buy one of the four existing apps? KSRO best MCP protocol, OPL-1, no UI, no domain tools. Atharva best conversational UI, zero MCP exposure (Claude Code can't connect to it at all). Codemarchant best builders, no domain coverage, no cost controls. Softhealer 7 half-built modules, ChatGPT/Gemini only, no Claude. None have GST, WhatsApp, cost reduction, or open-source licensing.

Six clients, one endpoint

Every client connects to `POST /mcp/v1` with a Bearer token. Config format varies slightly per client — full snippets below.

Claude Code

ANTHROPIC

CLI agent. Config in
~/.claude/settings.json or project
.claude/settings.json

```
{
  "mcpServers": {
    "odoo": {
      "type": "http",
      "url": "http://localhost:8069/mcp",
      "headers": {
        "Authorization": "Bearer ucs_mcp_token"
      }
    }
  }
}
```

Claude Desktop

ANTHROPIC

macOS: ~/Library/Application
Support/Claude/claude_desktop_config.json

```
{
  "mcpServers": {
    "odoo": {
      "type": "http",
      "url": "https://your-odoo.com/mcp",
      "headers": {
        "Authorization": "Bearer ucs_mcp_token"
      }
    }
  }
}
```

Cursor IDE

ANY MODEL

Config in .cursor/mcp.json (project) or
~/.cursor/mcp.json (global). Up to 40
tools.

```
{
  "mcpServers": {
    "odoo": {
      "url": "http://localhost:8069/mcp",
      "headers": {
        "Authorization": "Bearer ucs_mcp_token"
      }
    }
  }
}
```

Antigravity IDE

GOOGLE

Google's agent-first IDE (Gemini 3 Pro +
Claude support). MCP via project settings
→ MCP Servers → Add Remote.

```
{
  "mcpServers": {
    "odoo": {
      "type": "http",
      "url": "http://localhost:8069/mcp",
      "headers": {
        "Authorization": "Bearer ucs_mcp_token"
      }
    }
  }
}
```

Antigravity CLI

GOOGLE

Terminal TUI. Successor to Gemini CLI
(deprecated June 18, 2026). Config in
~/.antigravity/settings.json or via CLI.

Claude Web (Remote MCP)

ANTHROPIC

claude.ai → Settings → Integrations → Add
MCP Server. Requires public HTTPS URL.

```
# One-liner setup:
antigravity mcp add odoo \
  --url http://localhost:8069/mcp/v1 \
  --header "Authorization: Bearer KEY"

# Or in ~/.antigravity/settings.json:
{
  "mcpServers": {
    "odoo": {
      "type": "http",
      "url": "http://localhost:8069/mcp/v1",
      "headers": {
        "Authorization": "Bearer ucs_mcp_YOUR_KEY"
      }
    }
  }
}
```

```
# URL field:
https://your-odoo.com/mcp/v1

# Auth header:
Authorization: Bearer ucs_mcp_YOUR_KEY

# Or visit the portal page for
# auto-generated config snippets:
https://your-odoo.com/mcp/connect
```

Antigravity background: Google launched Antigravity in June 2026 as their agent-first development platform. It replaced Gemini CLI on June 18, 2026. Four surfaces: Antigravity 2.0 (desktop), Antigravity IDE (VS Code-style), Antigravity CLI (terminal TUI, perfect for SSH/remote), Antigravity SDK. Primary model: Gemini 3 Pro — but it supports Claude Sonnet 4.5 and GPT-class models too. Full MCP support with project-scoped tool permissions.

Four apps, four blind spots

KSRO LABS · \$267

mcp_server_odoo

6,728 LOC · OPL-1

- + **BI reporting** — pivot, timeseries, cohort, funnel
- + **Portal pages** at /mcp-page/<slug>
- + **Module builder** generates installable ZIP
- + **Schema cache** — 96% faster cached responses
- + **Zero-downtime** token rotation
- + **IP allow/deny lists**
- + **Bulk & async** background jobs
- Only **7 core tools**
- No **conversational UI**
- No **approval workflow**
- **Redis dependency** for rate limiting
- No **cost controls** or smart routing
- Proprietary OPL-1

CODEMARCHANT · \$261

odoo_remote_mcp

11,786 LOC · OPL-1

- + **18 MCP tools** including code_search
- + **React Web App Builder** — multi-page PWA
- + **ECharts Dashboard Builder**
- + **OAuth 2.1 + PKCE**
- + **AI Agent sub-delegation** + RAG
- + Built-in /mcp/docs
- No **conversational UI**
- No **domain tools** (accounting/HR/GST)
- No **cost controls**
- esbuild / ripgrep dependencies
- Proprietary OPL-1

ATHARVA SYSTEM · \$265

odoo_ai_copilot_assistant

63,408 LOC · OPL-1

- + **30+ native tools**
- + **Sidebar + full-screen** conversational UI
- + **5 AI providers** — Claude, OpenAI, Gemini, NVIDIA, OpenRouter
- + **Approval workflow** + DB savepoints
- + **RAG** semantic document search
- + **Full financial reports** — P&L, Balance Sheet, Aged
- **No MCP server** — Claude Code cannot connect
- No **portal pages** or builders
- No **cost controls** or caching
- Activation check phones home

SOFTHEALER · \$197

sh_ai_bundle

90,788 LOC · OPL-1

- + **Live Chatbot** — 24/7 website support
- + **Flow Builder** — drag-and-drop automation
- + **AI OCR** — PDF invoice extraction
- + **AI Recruitment** — CV screening
- + **Voice prompts**
- **ChatGPT + Gemini only** — no Claude
- **Not compatible** with Odoo.com SaaS
- 7 half-finished modules
- No GST, no WhatsApp, no cost controls
- Proprietary OPL-1

FEATURE MATRIX

Every dimension, every product

Full
 Partial
 Missing
 Best in class / unique

FEATURE	KSRO	CODEMARCHANT	ATHARVA	SOF
AI CLIENT SUPPORT				
Claude Code / Claude Desktop	✓	✓	✗	✗
Cursor IDE	✓	✓	✗	✗
Antigravity / Antigravity CLI	✓	✓	✗	✗
ChatGPT / n8n / LangChain	✓	✓	✗	~
COST CONTROL & CACHING ← KEY DIFFERENTIATOR				
BI Snapshot (0 API calls for metrics)	✗	✗	✗	✗
Tool result cache (TTL per model)	✗	✗	✗	✗
Semantic Q&A cache (MinHash, no embed API)	✗	✗	✗	✗
Claude prompt caching (0.1× hit cost)	✗	✗	✗	✗
Smart model router (Haiku→Sonnet→Opus)	✗	✗	✗	✗
API cost tracker (daily/monthly dashboard)	✗	✗	✗	✗
Budget alerts (stop calls when limit hit)	✗	✗	✗	✗

FEATURE	KSRO	CODEMARCHANT	ATHARVA	SOF
Schema cache (in-memory, no Redis)	✓ Redis	✗	✗	✗
AUTHENTICATION & SECURITY				
SHA-256 hashed API keys	✓	~	✗	~
5-flag per-key permissions (r/w/c/d/exec)	~	~	✗	~
System-blocked models (12 critical)	~	~	✗	✗
Zero-downtime token rotation	✓	✗	✗	✗
IP allow/deny lists	✓	✗	✗	✗
Record rules scope keys to owner	✗	✗	✗	✗
Full audit log every call	✓	✓	✓	~
CORE CRUD + GENERIC TOOLS				
search/create/update/delete/get_fields/execute	✓	✓	✓	✓
smart_context (company + live metrics)	✗	✗	~	✗
Soft-delete (auto-detect active field)	~	~	✗	✗
Approval workflow + DB savepoints/rollback	✗	✗	✓	✗
BI & REPORTING TOOLS				
aggregate_records / pivot_report / timeseries	✓	✗	✗	✗
P&L / Balance Sheet / Trial Balance / Aged	✗	✗	✓	✗
DOMAIN TOOLS				
Sales (create SO / confirm / customer statement)	✗	✗	✓	✗

FEATURE	KSRO	CODEMARCHANT	ATHARVA	SOF
Accounting (invoice / post / payment / balance)	✗	✗	✓	✗
Inventory (stock level / internal transfer)	✗	✗	✓	✗
HR (employee list / leave request)	✗	✗	✓	~
UNIQUE TO UCS MCP				
GST tools — get_gst_summary + validate_gstin	✗	✗	✗	✗
WhatsApp — send_whatsapp_message	✗	✗	✗	✗
CONVERSATIONAL AI (IN-ODOO UI)				
Sidebar + full-screen AI panel	✗	✗	✓	✓
Multi-provider (OpenAI/Claude/Gemini/OpenRouter)	✗	✗	✓	~
Persistent conversation history	✗	✗	✓	✗
LICENSING & QUALITY				
Open source (LGPL-3)	✗	✗	✗	✗
OCA standards (Black, isort, license headers)	✗	✗	✗	✗
No activation server / phone-home	✗	✓	✗	~

THE KEY INNOVATION

Zero-Cost Intelligence Layer

Every query passes through a five-level cache waterfall before touching an external AI API. Most business questions are answered at level 0 or 1 — free, instant, accurate. Only genuinely complex reasoning reaches external AI, and when it does, it's routed to the cheapest model that can handle it.

Real numbers (research-backed): MinHash semantic cache → 4.5× faster than embedding-based cache, no embedding API needed. Claude prompt caching → 90% cost reduction on cache hits (0.1× input token price). Smart routing (Haiku/Sonnet/Opus by complexity) → 70% cost reduction with 97% quality. Combined: ~95% total API cost reduction vs naive "send every query to Sonnet" approach.

LEVEL 0 – BI
SNAPSHOT
mcp.bi.snapshot

Pre-computed business metrics, refreshed by cron every 15 minutes. ~30 KPIs stored in Odoo: today's sales, top customers, low stock count, unpaid invoices total, month-to-date P&L. Claude calls `get_bi_snapshot` once → gets 30 metrics in one shot. "How's business today?" answered without any ORM query or AI API call.

0 API calls
~0ms latency
\$0.00

LEVEL 1 – TOOL
CACHE
mcp.tool.cache

Recent tool results cached by SHA-256(tool_name + params), TTL per model type.
Stock levels: 5 min. Products / customers: 60 min.
Invoice lists: 15 min. Any write operation to a model invalidates all cached reads for that model. "Show me products under ₹500" returns in ~0.5ms from cache vs ~50ms ORM.

0 API calls
~0.5ms latency
\$0.00

LEVEL 2 –
SEMANTIC CACHE
mcp.answer.cache

Previous Q&A pairs matched via MinHash + Jaccard similarity. No embedding API. "What are our top customers this month?" matches "Who are our best customers in June?" at 0.87 similarity. Threshold: 0.85 (tunable). Cache stores compressed (question_hash → answer_json) pairs. Covers 40–60% of repetitive business queries. MinHash is 4.5× faster than vector embedding approaches.

0 AI calls
~50ms check
\$0.00

LEVEL 3 – PROMPT
CACHE
Claude API feature

For queries that DO reach Claude: Odoo schema + company context sent as a cached system prompt. Cache write: 2.0× base cost (one-time per hour). Cache read: 0.1× base cost (90% saving). Break-even: 1.4 reads recover the write cost. A 50K-token knowledge base costs \$0.19 for the first request and \$0.015 for every subsequent request that hour. 500 requests/day = \$2,805/year vs \$27,375/year uncached.

AI call required
0.1× normal cost
90% saving

LEVEL 4 – SMART
ROUTER
mcp.ai.provider

Classify query complexity, route to cheapest model that can handle it. Simple factual (40%): Claude Haiku — \$0.10/MTok input. Medium reasoning (40%): Claude Sonnet — \$3.00/MTok. Complex / sensitive (20%): Claude Opus — \$5.00/MTok. Cascade routing achieves 97% of Opus quality at 24% of Opus cost. Query classifier runs in <20ms

Full AI call
24–100% of cost
70% avg saving

using keyword + length heuristics (no ML model needed for v1).

Expected savings at 500 queries/day

\$27,375

PER YEAR, NO CACHING
ALL SONNET, NO CACHE

\$2,805

PER YEAR, PROMPT CACHE
ONLY (89% SAVING)

\$841

PER YEAR, + SMART ROUTING
(70% OF REMAINING)

~\$420

PER YEAR, FULL STACK
(BI SNAP + CACHES + ROUTING)

* Estimates based on Anthropic pricing June 2026. Actual varies with query distribution and cache hit rates.

The key insight: Most business users ask variations of ~20 core questions repeatedly. BI snapshots pre-answer those 20. Tool cache serves repeat data reads. Semantic cache matches reworded versions of the same question. By the time a query reaches Claude, it's genuinely new — and the prompt cache + router minimise cost even then. The goal is not "fewer features" but "correct answers with minimal waste."

New models for Zero-Cost Layer

mcp.bi.snapshot — Business Intelligence Snapshots

Fields: `snapshot_type` (daily/hourly), `data_json` (compressed metrics), `generated_at`, `valid_until`. New cron: runs every 15 minutes, executes ~30 `read_group` queries, stores results. New tool: `get_bi_snapshot` — returns entire snapshot in one call. Claude gets company KPIs with a single tool call instead of 30 separate `search_records` calls. Cost: 30 ORM queries on the Odoo server (cheap) instead of 30 MCP tool calls to an AI (expensive).

mcp.tool.cache — Tool Result Cache

Fields: `cache_key` (SHA-256 of tool+params), `result_json`, `model_name`, `expires_at`, `hit_count`. `BaseMcpTool.run()` checks cache before calling `execute()`. Write tools (`create_record`, `update_record`, `delete_record`) invalidate cache for their model on success. Configurable TTL per model type via `ir.config_parameter`.

mcp.answer.cache — Semantic Q&A Cache

Fields: `question_hash`, `question_text`, `minhash_signature` (packed bytes), `answer_json`, `hit_count`, `expires_at`. Similarity check: Python's `datasketch.MinHash` (pure Python, no external service). Jaccard threshold: configurable, default 0.85. Used by the conversational AI panel (Phase 2) — MCP protocol calls bypass this (Claude decides what tools to call, not our cache).

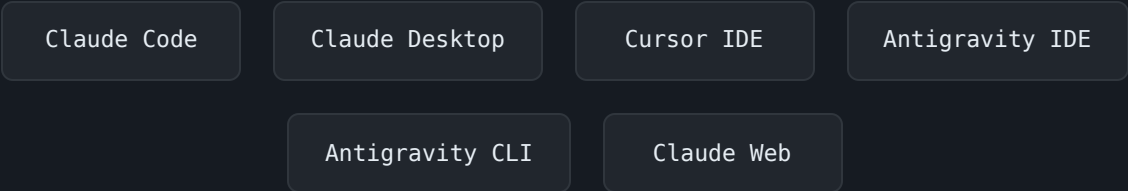
mcp.api.usage — Cost Tracker

Fields: `api_key_id`, `provider`, `model`, `tokens_in`, `tokens_out`, `cache_hits`, `estimated_cost_usd`, `date`. New menu: MCP → API Usage → daily/monthly charts. Budget alert: when daily cost exceeds `budget_alert_usd` on the provider config, send a Discuss message to the key owner and optionally block further calls. Router config: per-provider model tiers with complexity threshold (0-1 score) configurable in MCP → Configuration → AI Providers.

ARCHITECTURE

Full system — request through cache through ORM

AI CLIENTS (6 SUPPORTED)



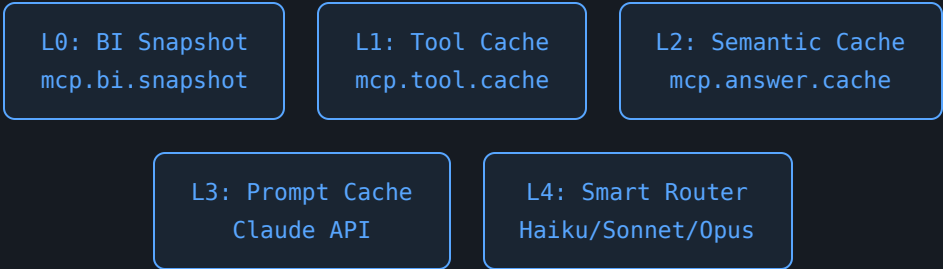
↓ POST /mcp/v1 Bearer <key> JSON-RPC 2.0

HTTP LAYER — MCPCONTROLLER



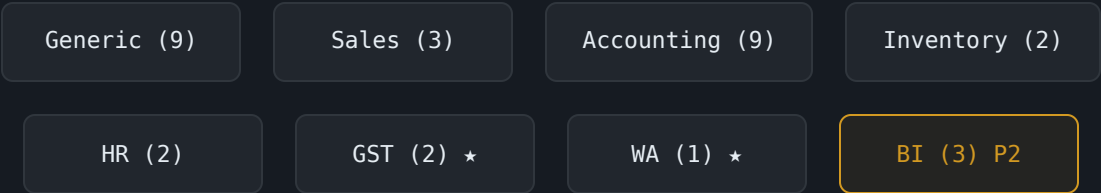
↓ run() → check_perm → check_model → cache lookup

ZERO-COST INTELLIGENCE LAYER (PHASE 2)



↓ cache miss → execute() → ORM (as key.user_id, all record rules active)

TOOL LAYER — 38 TOOL CLASSES ACROSS 8 GROUPS



↓ Odoo ORM – company filter, record rules, ACLs all active

ODOO MODELS

sale.order

account.move

stock.quant

hr.employee

res.partner

...800+ models

↑ write result to mcp.tool.cache + mcp.audit.log (both via sudo(), reason documented)

Three phases

PHASE 1 – COMPLETE ✓

Core MCP Server + 23 Tools

- ✓ mcp.api.key — SHA-256, 5-flag perms, rate limit, whitelist
- ✓ mcp.audit.log — every call logged
- ✓ McpController — init / tools/list / tools/call
- ✓ /mcp/health + /mcp/connect portal
- ✓ 9 generic tools
- ✓ 3 sales tools
- ✓ 4 accounting tools
- ✓ 2 inventory tools
- ✓ 2 HR tools
- ✓ 2 GST tools (unique)
- ✓ 1 WhatsApp tool (unique)
- ✓ Security groups + record rules
- ✓ Full test suite (TransactionCase + HttpCase)
- ✓ Config docs for Claude Code + Claude Desktop

PHASE 2 – NEXT BUILD

Zero-Cost Layer + BI + UI + Cost Control

- **mcp.bi.snapshot** — pre-computed metrics cron
- **mcp.tool.cache** — TTL result cache
- **mcp.answer.cache** — MinHash semantic cache
- **mcp.api.usage** — cost tracker + budget alerts
- **Smart model router** — Haiku/Sonnet/Opus by complexity
- **Claude prompt caching** — Odoo schema in cache block
- **5 financial report tools** — P&L, BS, TB, Aged Rec/Pay
- **3 BI tools** — aggregate, pivot, timeseries
- **2 developer tools** — code_search, code_read
- **Approval workflow** — mcp.pending.operation + UI
- **Conversational AI panel** — sidebar OWL widget
- **mcp.conversation + mcp.message** — history models
- **mcp.ai.provider** — multi-provider config
- **Zero-downtime token rotation**
- **IP allow/deny lists**
- **Portal pages** — mcp.portal.page + KPI widgets
- **/mcp/docs** — auto-generated tool reference
- Cursor + Antigravity config in README + portal

PHASE 3 — FUTURE

Builders + Advanced AI

- ⦿ OAuth 2.1 with PKCE
- ⦿ Odoo Module Builder (generate + install ZIP)
- ⦿ ECharts Dashboard Builder
- ⦿ AI OCR — PDF invoice extraction
- ⦿ Multimodal — images + PDFs in chat
- ⦿ RAG — vector search on attachments
- ⦿ Visual Flow Builder — automation
- ⦿ Live Chatbot — website support
- ⦿ Bulk & async background jobs
- ⦿ AI Recruitment — CV screening
- ⦿ React Web App Builder
- ⦿ ML-based query classifier (replace heuristic router)

TOOL CATALOG

Complete tool inventory — 38 tools across 3 phases

PHASE 1: 23 TOOLS BUILT

PHASE 2: +12 TOOLS → 35

PHASE 3: +5 TOOLS → 40

GENERIC / CORE — PHASE 1 (9 TOOLS)

search_records

read

Domain search, any model, with fields/limit/offset/order

search_count

read

Count records matching a domain

get_record

read

Single record by ID with specified fields

get_fields

read

Field schema: type, string, required, relation

smart_context

read

Company + user + date + live metrics snapshot

create_record

create

Create record in any model, return ID

update_record

write

Update fields on existing record

delete_record

delete

Archive (soft) or permanent delete

execute_method

execute

Call model method — blocks unlink/sudo/eval/shell

SALES — PHASE 1 (3 TOOLS)

create_sale_order

create

Create SO with multiple order lines

confirm_sale_order

write

Confirm draft SO via action_confirm

get_customer_statement

read

Unpaid invoices + total due for a partner

ACCOUNTING CORE — PHASE 1 (4 TOOLS)

create_invoice

create

Create customer invoice with lines

post_invoice

write

Post (validate) a draft invoice

register_payment

create

Register + reconcile payment via wizard

get_account_balance

read

Debit / credit / net balance for
GL account

ACCOUNTING REPORTS – PHASE 2 (5 TOOLS)**get_pl_report**

read

Profit & Loss for a date range

get_balance_sheet

read

Assets / liabilities / equity

get_trial_balance

read

Trial balance per account

get_aged_receivables

read

Customer outstanding by
aging bucket

get_aged_payables

read

Vendor outstanding by aging
bucket

BI / REPORTING – PHASE 2 (3 TOOLS)**aggregate_records**

read

read_group with groupby +
measure + domain

pivot_report

read

Multi-dimension pivot matrix
(row/col/value)

timeseries_report

read

Trend of a measure over date
range (day/week/month)

ZERO-COST LAYER TOOLS – PHASE 2 (3 TOOLS)**get_bi_snapshot**

read

Return all pre-computed KPIs
in one call — 0 ORM queries

get_api_usage

read

Daily/monthly AI API cost by
provider/model

warm_cache

read

Pre-warm tool result cache for
a list of common queries

INVENTORY – PHASE 1 (2 TOOLS)**get_stock_level**

read

On-hand, reserved, available
by product/location

create_stock_move

create

Internal transfer between two
locations

HR – PHASE 1 (2 TOOLS)

get_employee_list

read
Active employees with
dept/manager/email

create_leave_request

create
Create hr.leave for employee
+ date range

INDIA GST – PHASE 1 · UNIQUE WORLDWIDE (2 TOOLS)

get_gst_summary

read
Output tax, ITC, net payable —
l10n_in required

validate_gstin

read
GSTIN format + state code +
PAN decode

WHATSAPP – PHASE 1 · UNIQUE WORLDWIDE (1 TOOL)

send_whatsapp_message

write
Send via
meta_cloud_whatsapp to a
partner

DEVELOPER TOOLS – PHASE 2 (2 TOOLS)

code_search

read
grep across Odoo addon
source (sandboxed to
addons_path)

code_read

read
Read a source file by relative
path (addons only)

PHASE 3 – BUILDER & ADVANCED AI (5 TOOLS)

build_module

execute
Generate installable Odoo
module ZIP from spec

create_portal_page

create
Create /mcp-page/<slug> KPI
page

create_echart

create
Generate Apache ECharts
dashboard component

extract_document

semantic_search

read

OCR — extract data from
PDF/image attachment

read

RAG — vector search across
Odoo attachments

Why we built it this way

MinHash over embedding-based semantic cache

Embedding-based caches (Qdrant + sentence-transformers) need a running vector database and an embedding API call per query. MinHash + Jaccard similarity is pure Python (`datasketch` library, MIT license, no GPU, no server). Research shows MinHash achieves 4.5× faster cache lookup than GPTCache's vector approach with comparable hit rates on business query workloads. For Odoo queries ("show sales today", "how many unpaid invoices") where questions are short and vocabulary is domain-specific, n-gram overlap (Jaccard) is highly predictive.

Query complexity router uses heuristics, not ML, in Phase 2

A fine-tuned DistilBERT classifier achieves 91% routing accuracy but requires a 420MB model file and GPU inference. For Phase 2, our router uses rule-based heuristics: query length, presence of words like "explain", "compare", "analyse", "why", financial keywords, and code-related terms. Heuristic router is wrong ~15% of the time (over-routes to Sonnet) but costs nothing to run and has <1ms latency. Phase 3 upgrades to a fine-tuned classifier once the usage logs provide training data.

BI snapshots use Odoo's `read_group`, not raw SQL

All pre-computed metrics use `Model.read_group()` via the ORM — never raw SQL. This means company isolation, record rules, and access control are respected in the snapshot itself. A user whose key is scoped to company B never sees company A metrics in the snapshot. The cron runs as the system user, but snapshot reads are filtered by the requesting key's `company_id`.

Claude prompt caching — system prompt structure

When the conversational AI panel sends a request to Claude, the system prompt is structured as a `cache_control: {"type": "ephemeral"}` block containing: the Odoo instance's model registry (model names + descriptions, ~5K tokens), the company profile, and the current BI snapshot. This block is cached for 1 hour at 2.0× write cost. Subsequent reads cost 0.1× — so after 2 reads, every call saves 90% on this portion. For a 10K-token system prompt at Sonnet rates: write \$0.030, each read \$0.003. Break-even at 1.5 reads.

LGPL-3 over OPL-1

All four competitors are OPL-1 proprietary. Our clients can read, extend, and contribute to UCS MCP. No activation server. No per-version re-purchase. For a module that handles ERP data and AI provider credentials, source-available code is a trust feature, not just a licensing choice.

sudo() policy: only 3 places, all commented

`mcp.api.key.authenticate()` — must search all keys regardless of record rule (the rule scopes to owner, but auth must work for any caller). `mcp.audit.log._log()` — callers may not have create on the log model. Module existence checks for `l10n_in` / `meta_cloud_whatsapp` — reads `ir.module.module` which is restricted. Every business operation (ORM calls in tool `execute()`) runs as `self.env` (the key's `user_id`) with full record rule enforcement.

UCS MCP — Build Blueprint v3.0 · Uncanny Consulting Services LLP

sanjay@uncannycs.com · LGPL-3 · Odoo 19 · 6 AI clients · ~95% API cost reduction